□ Problem #SEPM-001: Lab Pratice Problem:

Comparing static and Non-static Variables in multi-threading.

Code:

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

/*
* Lab SEPM-001: static vs Non-static counters in multithreading.
*NOTE: Rename the class AND the file to YourFirstname_Thread.
* (e.g. class Rahim_Thread -> file Rahim_Thread.java)
*/
Public class Alif_Thread {

// EXPERIMENT A: thread-safe shared (static) counter
static final AtomicLong atomicsstatic = new
                    AtomicLong(0);
    static long unsafestatic =0;

static class CounterTask implements Runnable{
private final long increaments;
private final boolean threadSafe;
longtanceCount =0; // non-static (per object)
```

```
CounterTask(long increaments, boolean threadsafe) {
    this.increaments = increaments;
    this.threadsafe = threadsafe;
}
@Override
    public void run() {
for(long i=0; i<increaments; i++) {
    instanceCount++;
 if (threadsafe) {
    atomicstatic.incrementAndGet();
 } else {
        unsafestatic++;
    }
  }
 }
}
// Runs one experiment, returns {staticCount, NonstaticTota
static long[] runExperiment(int threads, long
  increments, boolean threadsafe) throws
        InterruptedException {
    atomicstatic.set(0);
    unsafestatic=0;
```

```
List<ThreadTask> tasks = new ArrayList<>();
List<Thread> pool = new ArrayList<>();
for (int i=0; i< threads; i++) {
CounterTask t = new CounterTask (increaments,
                                 threadsafe);
    task.add(t);
    pool.add(new Thread(t)); }
for (Thread t : pool) t.start();
for (Thread t : pool) t.join();
long nonstaticTotal = 0;
for (CounterTask t : tasks) nonstaticTotal += t.
                                 instanceCount;
long staticCount = threadsafe ? atomicStatic.get():
                                 unsafestatic;
return new long[] { staticCount, nonstaticTotal};
}
static string percent (long diff, long nonstatic) {
if (nonstatic == 0) return (diff == 0) ? "0.0000%"
                                 : "undefined";
return string.format("%.4f%%", (diff * 100.0) /
                                 nonstatic);
}
```

```
static void report(int threads, long increments,
        boolean threadsafe) throws
InterruptedException {
    long[] r = runExperiment(threads, increments,
                             threadsafe);
    long expected = (long) threads * increments;
    long diff = Math.abs(r[0] - r[1]);
    System.out.println("Mode      : " + (threadsafe ? "thread-
    saf (AtomicLong)" : "Unsynchronized (long)"));
    System.out.println("Threads (N) : " + threads);
    System.out.println("Increments/thread (k) : " +
                       increments);
    System.out.println("Expected (N*k) : " + expected);
    System.out.println("static count : " + r[0]);
    System.out.println("Non-static total : " + r[1]);
    System.out.println("Absolute difference : " + diff);
    System.out.println("percentage diff : " +
                       percent(diff, r[1]
}
```